

Git and GitHub Test Questions

1. What is Git and why is it used?

Ans: Git is a distributed version control system used for tracking changes in source code during software development. It allows multiple developers to work on a project simultaneously, provides tools for managing changes, and maintains the history of every modification. Git helps in collaboration, version tracking, and managing large projects with efficiency.

2. Explain the difference between Git pull and Git fetch.

Ans: **Git pull**: This command fetches changes from a remote repository and directly merges them into the current branch in the local repository.

Git fetch: This command retrieves changes from a remote repository but does not merge them. It updates the local references to the remote branches and allows you to review the changes before integrating them.

3. How do you revert a commit in Git?

Ans: To revert a commit in Git, we use the `git revert` command followed by the commit hash. This command creates a new commit that undoes the changes introduced by the specified commit. For example:
`git revert <commit-hash>`

4. Describe the Git staging area.

Ans: The Git staging area, also known as the index, is an intermediate area where changes are stored before they are committed to the local repository. It allows us to selectively stage changes and prepare them for the next commit, providing control over which modifications are included.

5. What is a merge conflict, and how can it be resolved?

Ans: A merge conflict occurs when Git cannot automatically resolve differences between merged branches because the same part of the code has been modified in different ways. To resolve a merge conflict, we must manually edit the conflicting files to select or combine the appropriate changes, then add the resolved files to the staging area and commit the merge.

6. How does Git branching contribute to collaboration?

Ans: Git branching allows developers to work on separate features, fixes, or experiments concurrently without affecting the main codebase. Each branch can be developed, tested, and reviewed independently. Once a branch is complete, it can be merged into the main branch. This workflow promotes collaboration, reduces conflicts, and ensures stability in the main codebase.

7. What is the purpose of Git rebase?

Ans: Git rebase is used to move or combine a sequence of commits to a new base commit. It helps in maintaining a linear project history by incorporating changes from one branch onto another. Rebase is often used to integrate changes from the main branch before merging feature branches, making the commit history cleaner and easier to understand.

8. Explain the difference between Git clone and Git fork.

Ans: **Git clone**: This command creates a local copy of an existing repository. It is used to download the entire repository, including its history, branches, and tags, to your local machine.

Git fork: Forking is a GitHub-specific feature that creates a personal copy of someone else's repository under our own GitHub account. It is typically used to propose changes to a project without affecting the original repository.

9. How do you delete a branch in Git?

Ans: To delete a branch in Git, we use the `git branch` command with the `-d` (safe delete) or `-D` (force delete) option, followed by the branch name. For example:

```
git branch -d <branch-name> # Safe delete
```

```
git branch -D <branch-name> # Force delete
```

10. What is a Git hook, and how can it be used?

Ans: A Git hook is a script that Git automatically runs before or after certain events, such as committing or pushing code. Hooks can be used to enforce policies, automate tasks, or integrate with other tools. Examples include pre-commit hooks to check code formatting or pre-push hooks to run tests. Hooks are stored in the `.git/hooks` directory of a repository.